The Cure and The Cause: Universal Function Approximation & the Curse of Dimensionality

Charlie Tharas

for the Williams College Math Colloquium on Nov. 11, 2026

1 Preamble

We focus on continuous, real valued functions on $K = [0,1]^d$.

An **input** is a list of d numbers

$$x = (x_1, \ldots, x_d) \in K$$

where $x_i \in [0,1]$

The **target function** is the 'rule' we want our network to imitate:

$$f : K \to \mathbb{R}$$

TODO: example of input & function; may want to roll with the same one throughout

Note: though our restriction of each input coordinate to $[0,1]$ may seem suspicious, consider the following:

- We can map any bounded input $x_i \in [a_i, b_i]$ to $[0,1]$ by normalizing: $z_i = \frac{x_i - a_i}{b_i - a_i}$, so choosing specific bounds $[0,1]$ here is mostly for convenience.
- It is admittedly true that many real-world inputs may be theoretically unbounded. However: i) we can often assume a reasonable practical bound (for instance, if approximating house prices as a function of square footage, we can probably assume no house will have over e.g. 1,000,000 sqft of space, or under 0), and ii) the role of boundedness here is mostly for mathematical convenience in our proof of a theoretical existence result; similar results (and, perhaps more importantly, empirical success!) exist for similar claims over unbounded inputs, but they are quite a bit trickier to discuss.

Though we specify the bounds of our K, in general it would suffice to take any compact subset of $\mathbb{R}^d$.

Theorem 1 (Heine-Borel)**.** $A \subseteq \mathbb{R}^d$ is compact iff. it is

- Closed: $\forall (x_n) \subseteq A.(x_n \to x \in \mathbb{R}^d \implies x \in A)$. No sequence of elements in A converges to a point outside the set, so approaching even a boundary of A cannot take us outside of it. This property differentiates $[0, 1]$ and $(0, 1)$.
- Bounded: $\exists R \in \mathbb{R}^+.\forall x \in A.\|x\|_2 \leq R$. Loosely, we can imagine that all of A can fit inside some ball of fixed diameter.

It should be easy to convince yourself that $K = [0, 1]^d$ is closed and bounded (and thus compact), as $\forall x \in K$, $0 \leq x_i \leq 1$ and $\|x\|_2 \leq \sqrt{d}$.[1]

Theorem 2 (Heine-Cantor)**.** Let $f : A \to B$ be continuous, with A a compact metric space and B a metric space. Then f is uniformly continuous, meaning that

$$\forall \varepsilon > 0.\exists \delta > 0.\forall (x, y \in A).(\|x - y\|_2 < \delta \implies |f(x) - f(y)| < \varepsilon)$$

Thus a continuous $f : K \to \mathbb{R}$ is uniformly continuous.[2]

Additionally, it is worth noting the difference between ordinary and uniform continuity:

- Ordinary continuity says that $\forall x \in K.\forall \varepsilon > 0.\exists \delta > 0.\forall y \in K.(\|x - y\|_2 < \delta \implies |f(x) - f(y)| < \varepsilon)$. In this statement, our tolerance may depend on x and ε.
- Uniform continuity says that $\forall \varepsilon > 0.\exists \delta > 0.\forall x, y \in K.(\|x - y\|_2 < \delta \implies |f(x) - f(y)| < \varepsilon)$. Here, a single scalar tolerance is applied to the entire space.

As an example, $f(x) = x^2$ is continuous on $\mathbb{R}$ but not uniformly continuous, as it becomes arbitrarily steep. However, on $[0, 1]$,

$$|x^2 - y^2| = |x - y||x + y| \leq 2|x - y|$$

lets us choose $\delta = \frac{\varepsilon}{2}$ for all points in the space, so $f(x)$ is uniformly continuous over $[0, 1]$.

[1] Recall that $\mathbb{R}^d$ is the familiar **Euclidean space** over d dimensions. This is the space of real coordinate vectors equipped with inner product $\langle x, y \rangle = \sum_{i=1}^{d} x_i y_i$ and distance $\|x - y\|_2 = \sqrt{\sum_{i=1}^{d} (x_i - y_i)^2}$.

[2] For our purposes it suffices to declare that K and $\mathbb{R}$ are metric spaces without rigorously proving why. Loosely speaking, we can think of a metric space as one with a reasonable notion of distance between points.

Why uniform continuity? We will use the fact that a continuous function on a compact domain is uniformly continuous to construct a grid over the domain. In particular, given an arbitrary error tolerance $\varepsilon > 0$, we choose $\delta > 0$ such that points less than δ apart have function values differing by less than ε. Then we can partition $[0,1]^d$ into finitely many cells, each with diameter $< \delta$, so that within any cell, the function differs from some representative point in that cell by less than ε, giving us a piecewise-constant approximation. It will require some extra steps to set up this construction on a continuous neural network, but we shall return to that problem later.

TODO: standard labeled visualization of neural network (will serve as anchor for the introduction of neurons/networks)

Our approximations are built with neural networks, which are in turn (ostensibly) built with **neurons**. A neuron takes a weighted combination of its inputs, shifts it, and applies a nonlinear function:

$$x \mapsto \sigma(w^\top x + b)$$

where $w^\top x = \sum_{k=1}^{d} w_k x_k$.

Here $w \in \mathbb{R}^d$ is the **weight vector**, $b \in \mathbb{R}$ is the **bias**, and $\sigma : \mathbb{R} \to \mathbb{R}$ is a nonlinear **activation function**.[3]

A **shallow** (1-layer) **neural network** is simply a connected collection of neurons:

$$N_\theta(x) = c + \sum_{j=1}^{m} a_j \sigma(w_j^\top x + b_j)$$

Where θ parametrizes the network and comprises all a_j, w_j, b_j, c. Note the newly defined a_j and c in our network, which parametrize an affine transformation of its neurons.

[3] While the affine transformation $w^\top x + b$ can be thought of as performing the bulk of the computation here, nonlinearity is also crucial, as it prevents multiple layers of neurons from collapsing. Recall that a composition of affine functions is affine, so a network of several affine layers is equivalent to some single-layer affine network. Regardless, we begin by discussing shallow networks with just one hidden layer, as universal approximation results hold for even such objects. We will later discuss the benefits of deeper networks.

Note: we traditionally refer to the weights and biases of the network as its **parameters** (this why you may hear people refer to ChatGPT as having "trillions of parameters"). Additional variables describing higher-level properties of the network (such as its shape) are **hyperparameters**, for instance, its width m, or its hidden layer count l (here in a shallow network, $l = 1$; we exclude our input layer x and output layer y from the count).

The goal of our network is to approximate the optimal function for our given task. The optimal function is selected from a **function class**

$$C(K) = \{f : K \to \mathbb{R} \mid f \text{ continuous}\}$$

As we are approximating some function from this class, we need a notion of comparison to say how close our approximation is. Traditionally, this role is assumed by the **uniform norm**

$$\|f\|_\infty = \sup_{x \in K} |f(x)|$$

Then, to say that our **approximation error** is less than ε for every input in K, we write

$$\sup_{x \in K} |f(x) - N_\theta(x)| = \|f - N_\theta\|_\infty < \varepsilon$$

Note: this definition already covers weaker notions of error or fit that may be familiar from e.g. statistics. For instance, you may have trained simple regression models (e.g. lines of best fit) on input probability distributions μ in an attempt to minimize the mean squared error (MSE)

$$\|f - N_\theta\|^2_{L^2(\mu)} = \int_K |f(x) - N_\theta(x)|^2 d\mu(x)$$

If our error is small everywhere, then the average error will be small. Thus we can immediately control this quantity with our notion of error described above, so we say it is 'stronger'.

2 Universal Function Approximation

2.1 One-dimensional approximation

To build intuition, we first consider $d = 1$, so our target is $f : [0, 1] \to \mathbb{R}$. Our goal is to build a network N_θ that approximates f uniformly.

First, consider how we might construct an approximation without a network. Take some continuous $f : [0, 1] \to \mathbb{R}$. Say we select $n + 1$ grid points:

$$0 = t_0 < t_1 < \cdots < t_n = 1$$

and sample our target function at these points, obtaining

$$(t_0, f(t_0)), \ldots, (t_n, f(t_n))$$

Assume we connect consecutive points with straight lines. This defines a piecewise function $p : [0, 1] \mapsto \mathbb{R}$ which linearly interpolates between our sampled points. In particular, the slope on the interval $[t_i, t_{i+1}]$ is

$$m_i = \frac{f(t_{i+1}) - f(t_i)}{t_{i+1} - t_i}$$

so

$$p(x) = f(t_i) + m_i(x - t_i)$$

for $x \in [t_i, t_{i+1}], i = 0, \ldots, n - 1$

TODO: concrete example: approximate x^2 by sampling $n : 2 \to$ big number; include visualization

We now formally provide a construction for a piecewise approximation of f.

Theorem 3. Let $f : [0, 1] \to \mathbb{R}$ be continuous and $\varepsilon \in \mathbb{R}^+$. We can construct a continuous piecewise-linear function $p : [0, 1] \to \mathbb{R}$ such that $\|f - p\|_\infty < \varepsilon$; in other words, p approximates f under some arbitrary error tolerance ε.

Proof. Fix arbitrary $\varepsilon > 0$. By uniform continuity, choose $\delta > 0$ such that

$$|x - y| < \delta \implies |f(x) - f(y)| < \frac{\varepsilon}{2}$$

By the Archimedean property, we choose $n \in \mathbb{N}$ with $1/n < \delta$.

Consider the evenly spaced grid

$$t_i = \frac{i}{n}, \quad i = 0, \ldots, n$$

Take any $i \in 0, \ldots, n - 1$ and $x \in [t_i, t_{i+1}]$, and let $\lambda = \frac{x - t_i}{t_{i+1} - t_i} \in [0, 1]$. Then, define

$$p(x) = (1 - \lambda)f(t_i) + \lambda f(t_{i+1})$$

We have $|x - t_i| < \delta, |x - t_{i+1}| < \delta$, so

$$|f(x) - f(t_i)| < \frac{\varepsilon}{2}, \quad |f(x) - f(t_{i+1})| < \frac{\varepsilon}{2}$$

Thus we have

$$\begin{aligned}|f(x) - p(x)| &= |(1-\lambda)(f(x) - f(t_i)) + \lambda(f(x) - f(t_{i+1}))| \\ &\leq (1-\lambda)|f(x) - f(t_i)| + \lambda|f(x) - f(t_{i+1})| \\ &\leq (1-\lambda)\frac{\varepsilon}{2} + \lambda\frac{\varepsilon}{2} \\ &= \frac{\varepsilon}{2}\end{aligned}$$

This holds throughout every interval $[t_i, t_{i+1}]$, so p is continuous and piecewise-linear, with

$$\|f - p\|_\infty \leq \frac{\varepsilon}{2} < \varepsilon$$

□

We now must show that we can build a neural network equivalent to p.

Recall that our network has m neurons, where the j-th neuron is $\sigma(w_j^\top x + b_j)$. Our parameters w_j, b_j are constructed (in practice, they are learned during training), while our nonlinear activation function σ is specified. Let

$$\sigma(x) = \text{ReLU}(x) = \max\{0, x\}$$

Note: A variety of activation functions have come into and out of popularity over the years. Selecting a good activation function requires considering its computability, training dynamics, and application-specific empirical performance. These days, ReLU is quite popular, owing to its simplicity and avoidance of "vanishing gradients"; it is also a convenient choice for our proofs here.
TODO: graph some activation functions? or not important enough

We can then define Shifted ReLU as

$$\sigma(x - t) = \begin{cases} 0 & \text{if } x \leq t \\ x - t & \text{otherwise} \end{cases}$$

This will come in handy momentarily.

We want our network's output to begin at height $f(0)$ and follow slope m_0, so our first neuron can compute

$$\sigma(x) = x \quad \text{on } [0, 1]$$

To do this, we can set the neuron's output weight to m_0 and the network's bias to $f(0)$. Then our network is described by

$$f(0) + m_0\sigma(x)$$

Our neuron reproduces p exactly from t_0 to t_1, but it continues with the same slope across the whole domain, whereas p may change slope at t_1. Since the next segment of p needs slope m_1, our overall function must change slope by $m_1 - m_0$.

To do this, we introduce a second neuron that computes $\sigma(x - t_1)$. This neuron contributes nothing to the final output for input below t_1, so it preserves the first neuron's contribution; after t_1, the neuron increases total output by slope $m_1 - m_0$. We define it as

$$(m_1 - m_0)\sigma(x - t_1)$$

So at this point, the overall network is described by

$$f(0) + m_0\sigma(x) + (m_1 - m_0)\sigma(x - t_1)$$

In the above network, for some input $> t_1$, the total slope is $m_0 + (m_1 - m_0) = m_1$, as required. Note that the output weight $m_1 - m_0$ can be negative if m_1 is less steep than m_0.

We can repeat this process, adding further neurons computing $\sigma(x - t_i)$ with output weight $(m_i - m_{i-1})$, for each segment of p. We can think of each neuron as 'switching on' at its particular grid point, and correcting the slope from that point onward.

Our final network is thus described by

$$N_\theta(x) = f(0) + m_0\sigma(x) + \sum_{i=1}^{n-1}(m_i - m_{i-1})\sigma(x - t_i)$$

Note that, although we constructed our neurons sequentially above, they all execute in parallel in our network's single hidden layer. As the resulting network starts at $f(0)$, remains continuous, and has the same slope as p on every interval, it is equivalent to p on $[0, 1]$. Thus

$$\|f - N_\theta\|_\infty = \|f - p\|_\infty < \varepsilon$$

TODO: visualization for this would probably be nice

2.2 Multidimensional Approximation

Extending the ideas we introduced above to multiple dimensions ($d > 1$) is nontrivial. In one dimension, a single grid point bifurcates the domain cleanly into everything 'before' and 'after' it. But in e.g. two dimensions, a ReLU neuron $\sigma(w_1x_1 + w_2x_2 + b)$ switches on over a whole line, with its effect extending across a half-plane. Thus a single ReLU neuron cannot cleanly make an isolated correction inside a single grid square; we will need different building blocks to mix information from different coordinates.

The key idea here will be to move from addition to multiplication. Consider restricting some arbitrary function to a small rectangle. We can define coordinate-wise restrictions $u(x_1)$, $v(x_2)$ such that u, v vanish outside their small intervals, but $u + v$ can remain nonzero if even one of the input coordinates is inside its respective interval. uv, however, vanishes when either coordinate is outside its allowed interval, allowing us to mix information from both coordinates!

TODO: example or (ideally) visualization

To motivate this idea for neural networks, we'll play around a bit with exponentials. Imagine that we replace ReLU with the exponential function, so that a single neuron becomes

$$x \mapsto e^{w^\top x}$$

The advantage of this is that, when multiplying two neurons, we get another exponential neuron

$$e^{u^\top x} e^{v^\top x} = e^{(u+v)^\top x}$$

This lets us rewrite a product of networks as an ordinary shallow network! For example, consider two networks

$$F(x) = a_1 e^{w_1^\top x} + a_2 e^{w_2^\top x}$$

$$G(x) = b_1 e^{v_1^\top x} + b_2 e^{v_2^\top x}$$

Multiplying and expanding gives

$$F(x)G(x) = a_1 b_1 e^{(w_1+v_1)^\top x} + a_1 b_2 e^{(w_1+v_2)^\top x} + a_2 b_1 e^{(w_2+v_1)^\top x} + a_2 b_2 e^{(w_2+v_2)^\top x}$$

Thus the product of two networks can be represented as one network with a single hidden layer, albeit with more neurons than either individual network. In fact, we will see that our argument for universal function approximation (rather unsatisfyingly) provides neither guarantees nor guidance on the number of neurons needed for our approximation.

Nevertheless, our reasoning above applies to any finite sums; for two networks with r and s neurons respectively, their product can be represented by at most rs neurons. Thus our family of functions is closed under multiplication. We will see why this is desirable momentarily.

Let us give a name to the family of finite weighted sums of exponential neurons:

$$\mathcal{E} = \{x \mapsto \sum_{j=1}^{m} a_j e^{w_j^\top x} \mid m \geq 1, a_j \in \mathbb{R}, w_j \in \mathbb{R}^d\}$$

We can view these as functions on our compact domain K. Importantly, our construction allows any finite width m, so increasing the number of neurons keeps us inside $\mathcal{E}$. We give this family a name to allow us to apply a very powerful theorem, which will serve as the 'hammer' of our proof.

Theorem 4 (Stone-Weierstrass)**.** Let K be compact and let $\mathcal{A} \subseteq C(K)$. Suppose that

- $\mathcal{A}$ contains every constant function
- $\mathcal{A}$ is closed under addition, multiplication by real scalars, and multiplication of functions
- $\mathcal{A}$ separates points; i.e. $\forall x, y \in K.(x \neq y \implies \exists h \in \mathcal{A} : h(x) \neq h(y))$

Then, for every $f \in C(K)$, for any $\varepsilon > 0$, $\exists h \in \mathcal{A}$ such that

$$\|f - h\|_\infty < \varepsilon$$

Broadly speaking, the hypotheses for Stone-Weierstrass ensure that our family of functions is rich enough to uniformly approximate every continuous target. In particular, we force $\mathcal{A}$ to separate points because if every available function gave identical outputs at two particular inputs, we could never approximate a target that assigned different values to those inputs. Under separation of points, a different function may distinguish every pair of inputs, allowing it to be approximated.

Proof. We apply Stone-Weierstrass to show that the family $\mathcal{E}$ uniformly approximates every continuous function on K.

Let $F, G \in \mathcal{E}$, with

$$F(x) = \sum_{j=1}^{r} a_j e^{u_j^\top x}, \quad G(x) = \sum_{k=1}^{s} b_k e^{v_k^\top x}$$

1. Each neuron $x \mapsto e^{w^\top x}$ is continuous, and finite weighted sums preserve continuity. Thus $\mathcal{E} \subseteq C(K)$

2. $\mathcal{E}$ contains every constant function. Choosing $w = 0$ gives $e^{0^\top x} = 1$, so giving a neuron output weight c produces $x \mapsto ce^{0^\top x} = c$.

3a. Addition combines the neurons of two networks into one larger hidden layer. In particular, $(F+G)(x) = \sum_{j=1}^{r} a_j e^{u_j^\top x} + \sum_{k=1}^{s} b_k e^{v_k^\top x}$, a finite weighted sum of at most $r + s$ exponential neurons. Since $F + G \in \mathcal{E}$, $\mathcal{E}$ is closed under addition.

3b. Multiplying by a scalar rescales the output weights of our neurons. In particular, $(cF)(x) = \sum_{j=1}^{r} ca_j e^{u_j^\top x} \in \mathcal{E}$, so $\mathcal{E}$ is closed under scalar multiplication.

3c. As we have already seen, multiplying two networks produces a finite sum of at most rs exponential neurons, so $\mathcal{E}$ is closed under multiplication of functions.

4. Take $x, y \in K$ such that $x \neq y$. WLOG, let them differ at coordinate k, so $x_k \neq y_k$. Choose a neuron whose weight vector 'selects' that coordinate, i.e.

the vector with its k-th coordinate equal to 1 and all others equal to 0. This neuron computes

$$x \mapsto e^{x_k}$$

Because the exponential is strictly increasing, $e^{x_k} \neq e^{y_k}$. Thus $\mathcal{E}$ separates points.

By conditions 1-4, our family of exponential neurons $\mathcal{E}$ satisfies the Stone-Weierstrass hypotheses. Thus for every continuous target f and every $\varepsilon > 0$, there exists a finite exponential network H such that

$$\|f - H\|_\infty < \frac{\varepsilon}{2}$$

□

We deliberately reserve half of our 'error allowance' here as H is currently comprised of exponential neurons. We will need to convert H back to ReLU neurons, which will introduce further approximation error.

To understand this conversion, let us begin with a single neuron from H:

$$x \mapsto e^{w^\top x}$$

Although x has many coordinates, this neuron uses them only to calculate one real number, $t = w^\top x$. After that, it merely applies the one-dimensional function $t \mapsto e^t$.

Because K is compact, $w^\top x$ stays in a bounded interval as x ranges over K. We can choose $A < B$ containing all of those values, so that we only need to approximate the exponential on $[A, B]$. In particular, we select an arbitrary error tolerance $\eta > 0$ and, by one-dimensional approximation, construct a one-dimensional ReLU network q such that

$$|e^t - q(t)| < \eta \quad \forall t \in [A, B]$$

> *Note:* our earlier construction applies to any finite interval by rescaling: we may set $s \mapsto \frac{t-A}{b-A}$, approximate $s \mapsto e^{A+(B-A)s}$ on $[0, 1]$, and then substitute back.

This construction is thankfully relatively straightforward. If a 'regular' neuron in q is

$$t \mapsto \sigma(\alpha t + \beta)$$

then after substituting $t = w^\top x$, the neuron becomes

$$x \mapsto \sigma(\alpha w^\top x + \beta) = \sigma((\alpha w)^\top x + \beta)$$

an ordinary ReLU neuron receiving the original input x. Thus $q(w^\top x)$ approximates the exponential neuron using only ReLU neurons in a single hidden layer.

Recall that our exponential network is described by

$$H(x) = \sum_{j=1}^{m} a_j e^{w_j^\top x}$$

To convert our whole network to ReLU, for each j we construct a one-dimensional ReLU approximation q_j on the interval containing all values of $w_j^\top x \ \forall x \in K$. Replacing every exponential gives

$$N_\theta(x) = \sum_{j=1}^{m} a_j q_j(w_j^\top x)$$

As each $q_j(w_j^\top x)$ is a shallow ReLU network, combining the hidden neurons and collecting the weighted output biases of all q_j gives one finite shallow ReLU network. We are almost at our final construction: the last step is to consider how accurately each q_j must approximate its exponential. To do this, we set η to control our final error tolerance, accounting for the fact that each replacement error is multiplied by its original output weight a_j.

If every replacement network q_j has approximation error $\leq \eta$, then

$$\begin{aligned} |H(x) - N_\theta(x)| &\leq \sum_{j=1}^{m} |a_j||e^{w_j^\top x} - q_j(w_j^\top x)| \\ &\leq \sum_{j=1}^{m} |a_j|\eta \\ &= \eta \sum_{j=1}^{m} |a_j| \end{aligned}$$

Let

$$\eta = \frac{\varepsilon}{2(1 + \sum_{j=1}^{m} |a_j|)}$$

This ensures that

$$\|H - N_\theta\|_\infty < \frac{\varepsilon}{2}$$

Then by the triangle inequality, our final approximation error is

$$\|f - N_\theta\|_\infty \leq \|f - H\|_\infty + \|H - N_\theta\|_\infty < \frac{\varepsilon}{2} + \frac{\varepsilon}{2} = \varepsilon$$

This completes our conversion back to ReLU.

2.3 Recap

We have shown that for every fixed finite input dimension d, every continuous target f on compact $K \subseteq \mathbb{R}^d$, and every $\varepsilon > 0$, there exists a finite ReLU network with one hidden layer whose uniform approximation error is less than ε.

Take a moment to appreciate what it took to get here:

- We showed that a shallow network of ReLU neurons can approximate any continuous f over compact K in one dimension
- To move to arbitarily many dimensions, we needed to use a shallow network of exponential neurons
- To connect this back to ReLU, we showed that a shallow ReLU network can approximate an exponential neuron, and thus a shallow exponential network can be approximated with a shallow ReLU network
- Stringing these together has allowed us to approximate any continuous f over compact K with finite dimensionality, in other words, we have demonstrated that a shallow ReLU network is a universal function approximator!

TODO: can qualify/spruce the above a bit more

TODO: we currently don't formalize this as a proof, so funnily enough this doc doesn't actually technically have a proof of UFA in it. Do we need one?

3 The Curse of Dimensionality

TODO: need to think about how much room I have and what content might be good to cover, particularly ideas relevant to what I have discussed above

TODO: at a high level, what I want to do with this section is provide some interesting commentary on why this existence result is utterly useless and why shallow networks fail to scale. I want to motivate why we want networks to be deeper and have some fun visualizations a la. Welch Labs [https://www.youtube.com/watch?v=qx7hirqgfuU] about how successive layers can be thought of as bifurcating the data plane into increasingly complex shapes. This will be less mathematically rigorous than the first section, a bit more visual, and considerably shorter. Probably good to discuss with Prof. before moving fwd